Junit5 Architecture

Junit4:

This Monolithic jar, is responsible for everything:

- Test Discovery
- Test Execution
- Annotations
- Reporting

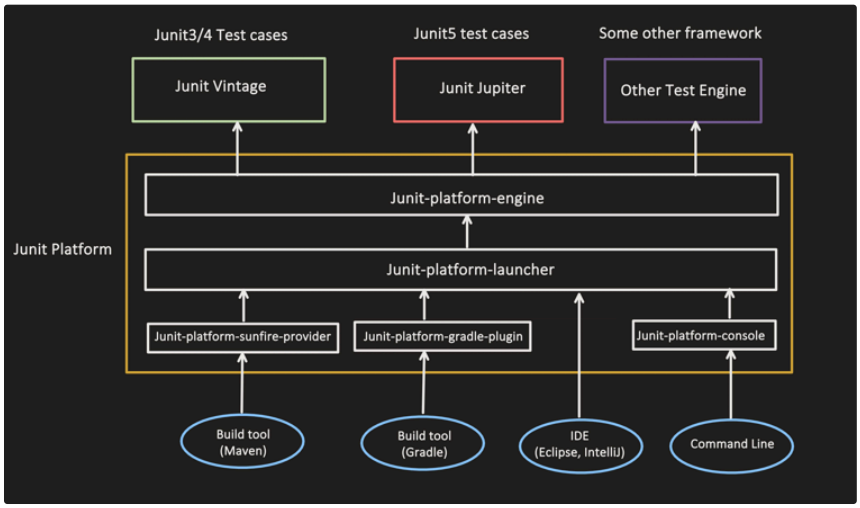
Concept && Coding

Etc...

And currently Junit4 is in "Maintenance Mode" and it is recommended to use Junit5 for new development.

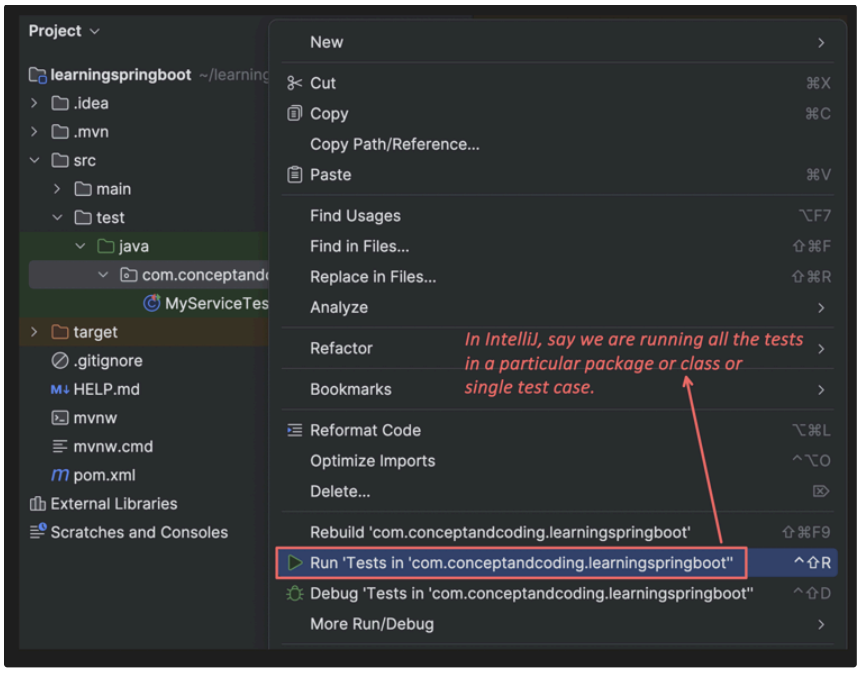
Junit5:

Composed of Multiple Modules



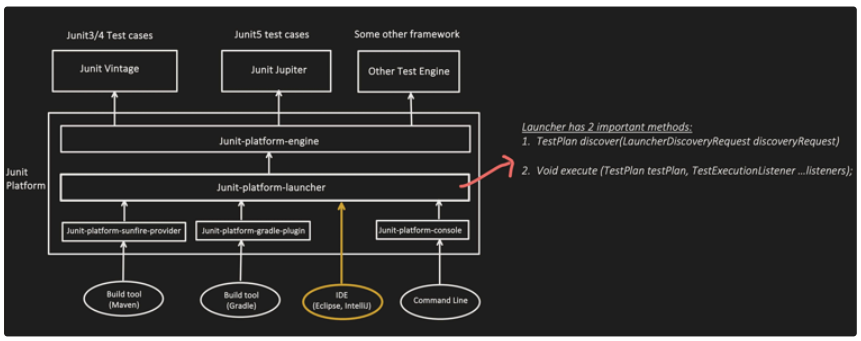
Lets understand this complete Architecture flow:

Step1: running a test case through IntelliJ



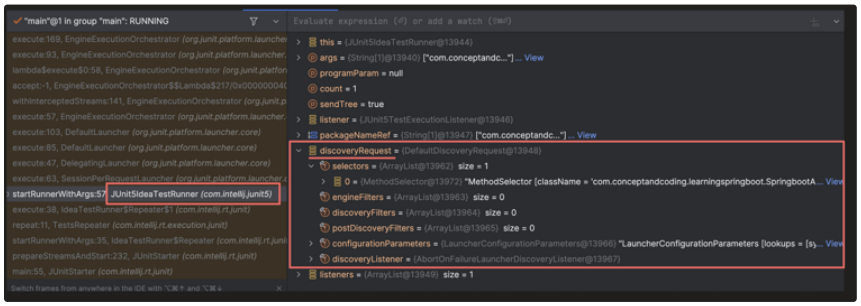
Step2: IntelliJ directly talks to Launcher, and initiate Discover() phase

Concept && Coding



Since, IntelliJ directly talks with Launcher means, IntelliJ has the code to invoke the Junit Launcher.

If we debug the call hierarchy, you can see "JUnit5IdeaTestRunner" class.
This is IntelliJ framework code. Its creating the DiscoveryRequest and invoking Junit Platform Launcher Discover API.

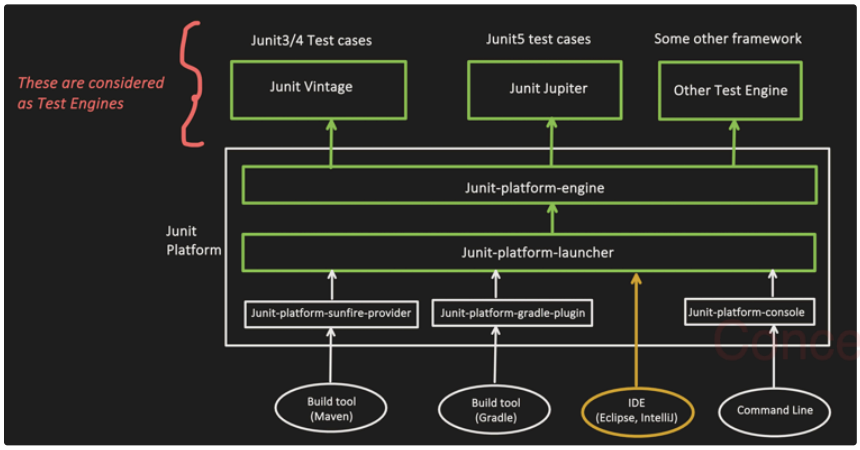


So, internally IntelliJ "*JUnit5IdeaTestRunner*" might be doing:

```
//Specify what tests to run (packages, classes etc.)
LauncherDiscoveryRequest request =
LauncherDiscoveryRequestBuilder.request()

.selectors(selectPackage("com.conceptandcoding.learningspringboot"))
```

```
                                                                .build();
//Creates an instance of Launcher
Launcher launcher = LauncherFactory.create();

//Invokes discover() method of Junit Launcher
TestPlan testPlan = launcher.discover(request);

```

Step3: Laucher acts as a coordinator and passes the discover() request to all registered TEST ENGINES, so each can discover and filter out their test cases:



Concept && Coding

Junit-platform-engine:

Provides an SPI (Service Provider Interface)

Exposes interface, which different Test engine (Vintage, Jupiter etc.) implement and integrate with Junit Platform:

```
interface TestEngine {

     TestDescriptor discover(EngineDiscoveryRequest request, UniqueId id);

      void execute(ExecutionRequest request);

}
```

Junit-platform-launcher:

Launcher methods:

1. *TestPlan discover(LauncherDiscoveryRequest discoveryRequest)*
2. *Void execute (TestPlan testPlan, TestExecutionListener ...listeners);*

In Discover() phase:

- Launcher receives a LauncherDiscoveryRequest with selector (package name, class name etc.)
- Launcher delegates this request to all the Test Engines (Jupiter, Vintage, other framework etc.)

```
//Launcher iterates and call the discover method for each Test engine.
for (TestEngine engine : testEngines) {
    TestDescriptor descriptor = engine.discover(...);
```

```
    root.addChild(descriptor);
}

```

- Each Test Engines discover and filter out their framework Test cases.

  Say in that package, we have multiple framework test cases like Junit4, Junit5, Cucumber etc.

  so Junit5(Jupiter) Test Engine will take out its framework test cases.

Class A:

- Method1
- Method2

  Class B:

- Method1
- Method2
- Method3

Concept && Coding

Similarly Junit4 (Vintage) Test Engine will take out its framework test cases.
Class C:

- Method1
- Method2

Class D:

- Method1

Once Launcher gets the response from all the Test Engines, it creates a single TestPlan out of it:

TestPlan =
Jupiter Framework test cases:

Class A:

- Method1
- Method2

Class B:

- Method1
- Method2
- Method3

Vintage Framework test cases:

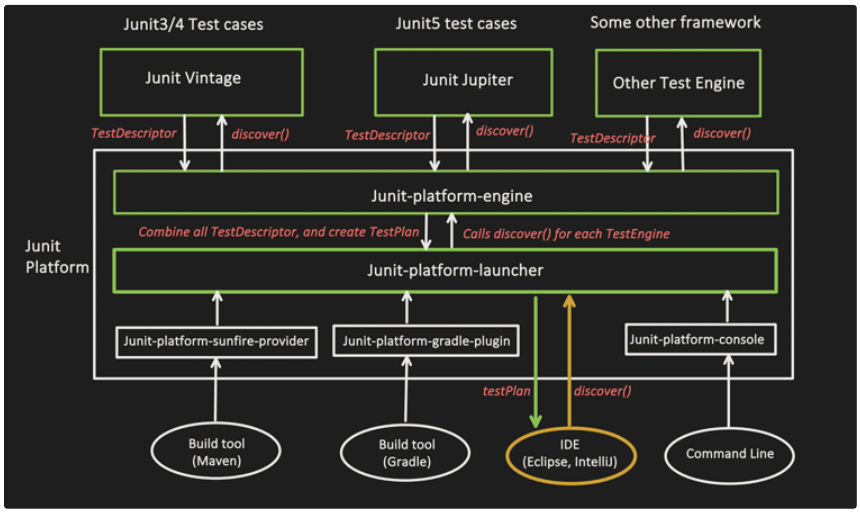
Class C:

- Method1
- Method2

Class D:

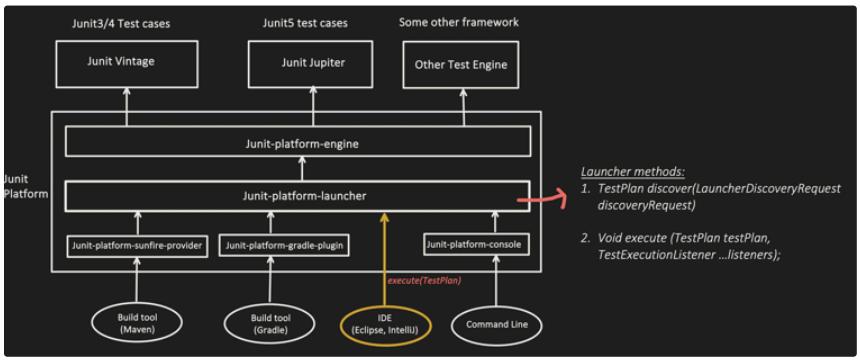
- Method1

And return it back to caller (IntelliJ)



Concept && Coding

Step4: Now, intelliJ will call execute() method of the Launcher




```
LauncherDiscoveryRequest request =
LauncherDiscoveryRequestBuilder.request()

.selectors(selectPackage("com.conceptandcoding.learningspringboot"))
                                                        .build();

Launcher launcher = LauncherFactory.create();
TestPlan testPlan = launcher.discover(request);

 launcher.execute(request);

```

Step5: Again, Launcher act as a coordinator and tells each Test Engine to run only its own tests.

During Execution:

- Launcher.execute(TestPlan, ...) is called.

- The TestPlan contains a **tree of TestDescriptors**, each tied to its corresponding **TestEngine**.

TestPlan =

Jupiter Framework test cases (Pass below TestDescriptor to Jupiter Test Engine):

Class A:

- Method1
- Method2

Class B:

- Method1
- Method2
- Method3

Vintage Framework test cases (This TestDescriptor will be passed to Vintage Test Engine):

Class C:

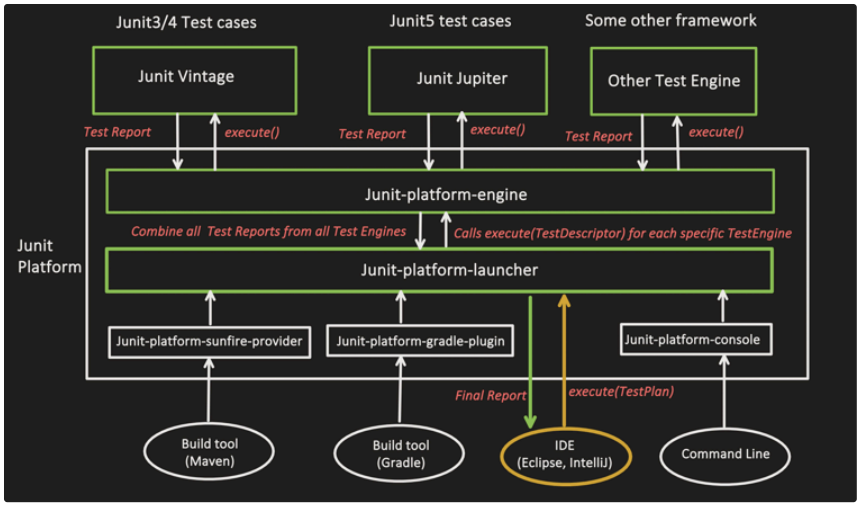
- Method1
- Method2

Class D:

Concept && Coding

- Method1

- Invokes **TestEngine.execute(ExecutionRequest)** for each engine **with only its relevant subtree**.
- Each Test Engine will return the report of test case pass and failure, will Launcher will combine and return back to IntelliJ.



With this, we also get to know the main advantage of this Architecture is:

- Multiple Test framework can be supported.

- New test cases can be written in Junit5 and old Junit4 test cases Migration can go in parallel, as Vintage Test Engine help to run Junit 4 (legacy) test cases on latest framework too.

- Parallel test (at Test engine level or withing an engine) is supported.

- Any New TestEngine will be supported through all means like maven, gradle, IntelliJ, Eclipse etc.. Thanks to Launcher, which act as a mediator.

Concept && Coding